

Discussion sur les attentes et les mécanismes de verrouillage des moniteurs

Fabrice.Kordon@lip6.fr

En guise d'introduction...

2



Question à l'examen de janvier 2018

- Soit une classe `Attendre` disposant d'une variable booléenne `dispo`. Cette classe dispose aussi d'une méthode `attendreDispo()`. Deux implémentations de cette méthode sont proposées. Expliquez précisément les différences entre ces deux implémentations.

```
public synchronized void attendreDispo1() throws InterruptedException {  
    while (! dispo) {  
        this.wait();  
    }  
    dispo = false;  
}
```

```
public synchronized void attendreDispo2() throws InterruptedException {  
    while (! dispo) {  
        Thread.sleep(100);  
    }  
    dispo = false;  
}
```

En guise d'introduction...

2

Question à l'examen de janvier 2018

 Soit une classe `AttendreDispo` disposant d'une variable booléenne `dispo`. Cette classe implémente la méthode `attendreDispo()`. Deux implémentations sont proposées. Précisez précisément...

Sur `attendreDispo1()`...

Attente passive...

```
public synchronized void attendreDispo1() throws InterruptedException {  
    while (!dispo) {  
        this.wait();  
    }  
    dispo = false;  
}
```

Sur `attendreDispo2()`...

Attente semi-active...

```
public synchronized void attendreDispo2() throws InterruptedException {  
    while (!dispo) {  
        Thread.sleep(10);  
    }  
    dispo = false;  
}
```

Mais pourquoi?

3

Attention aux verrous

- Les méthodes sont synchronisées
 - ▶ Idem avec un `synchronized(this) { ... }` dans le corps

Différence entre `wait()` et `sleep(...)`?

- Le premier relâche le verrou
- Le second ne relâche pas le verrou...

Conclusion?

- `attendreDispo1()` est une attente passive
- `attendreDispo2()` ne fonctionne pas!!!

Vérifions sur le code

4

```
class Attendre {  
    private boolean dispo = false;  
  
    public synchronized void attendreDispo1() throws InterruptedException {  
        while (! dispo) {  
            this.wait();  
        }  
        dispo = false;  
    }  
  
    public synchronized void attendreDispo2() throws InterruptedException {  
        while (! dispo) {  
            Thread.sleep(100);  
        }  
        dispo = false;  
    }  
  
    synchronized void setDispo() {  
        dispo = true;  
        this.notifyAll(); // pour attendreDispo1  
    }  
}
```

Vérifions sur le code

4

```
class U implements Runnable {
    Attendre monA;

    U (Attendre a) {
        monA = a;
    }

    public void run() {
        try {
            System.out.println ("U attend");
            monA.attendreDispo1();
            // monA.attendreDispo2();
            System.out.println ("U se termine");
        } catch (Exception e) {
            System.out.println ("levée de "+e);
        }
    }
}
```

Vérifions sur le code

4

```
public class DesAttentes {  
    public static void main(String []args) {  
        Attendre a = new Attendre();  
        Thread t = new Thread(new U(a));  
        System.out.println ("Main lance un U");  
        t.start();  
        try {  
            Thread.sleep(600);  
        } catch (Exception e) {  
            System.out.println ("levée de "+e);  
        }  
        a.setDispo();  
    }  
}
```

Exécution...

5

Avec attendreDispo1()

```
fko $ javac -Xlint DesAttentes.java ; java DesAttentes
Main lance un U
U attend
U se termine
```

Avec attendreDispo2()

```
$ javac -Xlint DesAttentes.java ; java DesAttentes
Main lance un U
U attend
^C
```

Et l'attente semi-active?

6

```
class Attendre {  
    private boolean dispo = false;  
  
    public void attendreDispo3() throws InterruptedException {  
        while (!this.testAndSetDispo()) {  
            Thread.sleep(100);  
        }  
    }  
  
    synchronized boolean testAndSetDispo () {  
        if (dispo) {  
            dispo = false;  
            return true;  
        } else {  
            return false;  
        }  
    }  
  
    synchronized void setDispo() {  
        dispo = true;  
        this.notifyAll(); // pour attendreDispo1  
    }  
}
```

Et l'attente semi-active?

6

class Attente {



Atomicité

Il faut la maîtriser

ows Inter
(()) {



Rappel!

Déjà vu: TaS (et CaS)

```
synchronized boolean testAndSetDispo () {  
    if (dispo) {  
        dispo = false;  
        return true;  
    } else {  
        return false;  
    }  
}
```

```
synchronized void setDispo() {  
    dispo = true;  
}
```

```
fko $ javac -Xlint DesAttentes.java ; java DesAttentes  
Main lance un U  
U attend  
U se termine
```

Est-ce lié au moniteurs des objets Java?

7

 À votre avis?



Et bien non!!!

Lié au concept de
moniteur

 En voici la preuve...

Avec la classe ReentrantLock

8

```
import java.util.concurrent.locks.*;
```

```
class Attendre {
```

```
    private ReentrantLock l = new ReentrantLock();
```

```
    private Condition c = l.newCondition();
```

```
    private boolean dispo = false;
```

```
    public void attendreDispo1() throws InterruptedException {
```

```
        l.lock();
```

```
        try {
```

```
            while (!dispo) {  
                c.await();
```

```
            }
```

```
            dispo = false;
```

```
        } finally {
```

```
            l.unlock();
```

```
        }
```

```
    }
```

```
    void setDispo() {
```

```
        l.lock();
```

```
        try {
```

```
            dispo = true;
```

```
            c.signalAll(); // attendreDispo1
```

```
        } finally {
```

```
            l.unlock();
```

```
        }
```

```
    }
```



Fonctionne...

Comme avant

Avec la classe ReentrantLock

8

```
public void attendreDispo2() throws InterruptedException {  
    l.lock();  
    try {  
        while (!dispo) {  
            Thread.sleep(100);  
        }  
        dispo = false;  
    } finally {  
        l.unlock();  
    }  
}
```



Plante...

Comme avant

Avec la classe ReentrantLock

8

```
public void attendreDispo3() throws InterruptedException {  
    while (!this.testAndSetDispo()) {  
        Thread.sleep(100);  
    }  
}
```

```
boolean testAndSetDispo () {  
    l.lock();  
    try {  
        if (dispo) {  
            dispo = false;  
            return true;  
        } else {  
            return false;  
        }  
    } finally {  
        l.unlock();  
    }  
}
```



Fonctionne...

Comme avant

Encore une chose...

9

```
import java.util.concurrent.locks.*;

class Attendre {
    private ReentrantLock l = new ReentrantLock();
    private Condition c = l.newCondition();

    private boolean dispo = false;

    public void attendreDispo2bis() throws InterruptedException {
        l.lock();
        try {
            while (!dispo) {
                l.unlock(); // relacher le verrou explicitement
                Thread.sleep(100);
                l.lock(); // reprendre le verrou explicitement
            }
            dispo = false;
        } finally {
            l.unlock();
        }
    }

    void setDispo() {
        l.lock();
        try {
            dispo = true;
            c.signalAll(); // attendreDispo1
        } finally {
            l.unlock();
        }
    }
}
```

Encore une chose...

9

```
import java.util.concurrent.locks.*;

class Attendre {
    private ReentrantLock l = new ReentrantLock();
    private Condition c = l.newCondition();

    private boolean dispo = false;

    public void attendreDispo2bis() throws InterruptedException {
        l.lock();
        try {
            while (!dispo) {
                l.unlock(); // relacher le verrou explicitement
                Thread.sleep(100);
                l.lock(); // reprendre le verrou explicitement
            }
            dispo = false;
        } finally {
            l.unlock();
        }
    }

    void setDispo() {
        l.lock();
        try {
            dispo = true;
            c.signalAll(); // attendreDispo1
        } finally {
            l.unlock();
        }
    }
}
```



Important

Impossible à faire avec
les moniteurs d'objets en Java

En guise de conclusion...

10

Ce qui est important

- Dans un moniteur, un seul fil d'exécution **actif** à la fois
 - ▶ Endormi $\neq$ suspendu
- Il faut donc maîtriser le verrouillage/déverrouillage
 - ▶ sleep ne déverrouille pas
 - ▶ wait/await déverrouille

Mais encore...

- Thread/processus actif $\neq$ élu

